

JSSP GA Project — Backlog

Small, deliberately-deferred items to come back to once the core pipeline (parser → chromosome → SBA → GA loop → experiments) is working end to end. None of these block progress right now — they matter for correctness/robustness and for the “clean code” portion of the grading before final submission.

1. Parser robustness (instance.py)

- Add comment-line skipping (lines starting with '#') to the parser. Currently deferred — testing only against a comment-free copy of la01.txt.

Risk flag: the assignment requires using the official JSPLib instance files, and the canonical files on GitHub DO include '#' comment headers. Test against

<https://raw.githubusercontent.com/tamy0612/JSPLIB/master/instances/la01> before final submission, or the parser may break on the actual required data.

- Validate that each job line splits into exactly $2 \times n_{\text{machines}}$ integers. On mismatch, raise a clear error naming the job index and what was found (caught the '1 49' → '149' typo bug by eye this time — won't always be that lucky).
- Validate the file isn't empty after stripping comments/blank lines; raise a clear error rather than proceeding with no data.
- Defensive check: after parsing, confirm the maximum machine index seen across the whole file equals $n_{\text{machines}} - 1$. Catches file corruption or off-by-one issues immediately instead of downstream in the SBA.
- Fix the hardcoded relative path (e.g. "../data/la01.txt") inside the parsing script. Pass the path in as a function argument from the caller instead, so the script doesn't silently break when run from a different working directory.

2. Testing

- Add a couple of pytest unit tests: (a) parser produces the expected $n_{\text{jobs}}/n_{\text{machines}}$ /job list on a known-good instance; (b) SBA decoder reproduces the hand-worked 2-job/2-machine example (expected makespan = 7); (c) active decoder's makespan is never worse than semi-active's on the same chromosome.

3. Documentation

- Add inline docstrings explaining the operation-based chromosome representation (why any permutation of the right job-ID multiset is automatically valid).
- Add inline docstrings explaining the SBA logic (semi-active vs. active, and why active can only match or beat semi-active).
- Write the README.md installation + usage sections properly (currently just section headers) — required by the submission spec.

4. Nice-to-have / stretch

- Upgrade from semi-active to active SBA once semi-active is fully verified.
- Gantt chart rendering (matplotlib) for at least one best-found schedule per instance category.
- Optional: mention Giffler-Thompson as a considered-but-not-implemented alternative in the report's discussion section, for extra conceptual breadth.